First, solve the problem. Then, write the code.

– John Johnson



@nixCraft

Automata and machines

Paolo Burgio
paolo.burgio@unimore.it



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

High Performance
Real Time Lab

Industrial embedded systems

What they do

- Monitor physical properties of the system/plant (via *sensors*)
- Might perform some control, or part of, control algos
- Via *actuators*

Control can be

- Continuous in time
- Discrete in time

➔ Control theory

Industrial controls in a nutshell



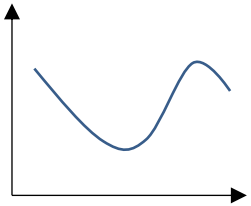
In their generic form,

$$F: \{ S, I \} \rightarrow \{ O \}$$

computed …when?

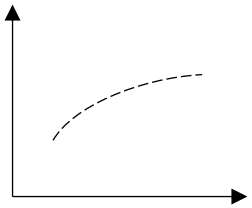
If continuous

- Physical properties and actuators are continuous in time
- *F(t)* continuous
- Combinatorial logic/analogic systems



If discrete

- Computed at pre-determined instance in time
- Event-driven (e.g., timeout, interrupt)
- Sequential logics/digital systems



Finite state automations for discrete controls

E.g., an elevator, reacts to multiple events

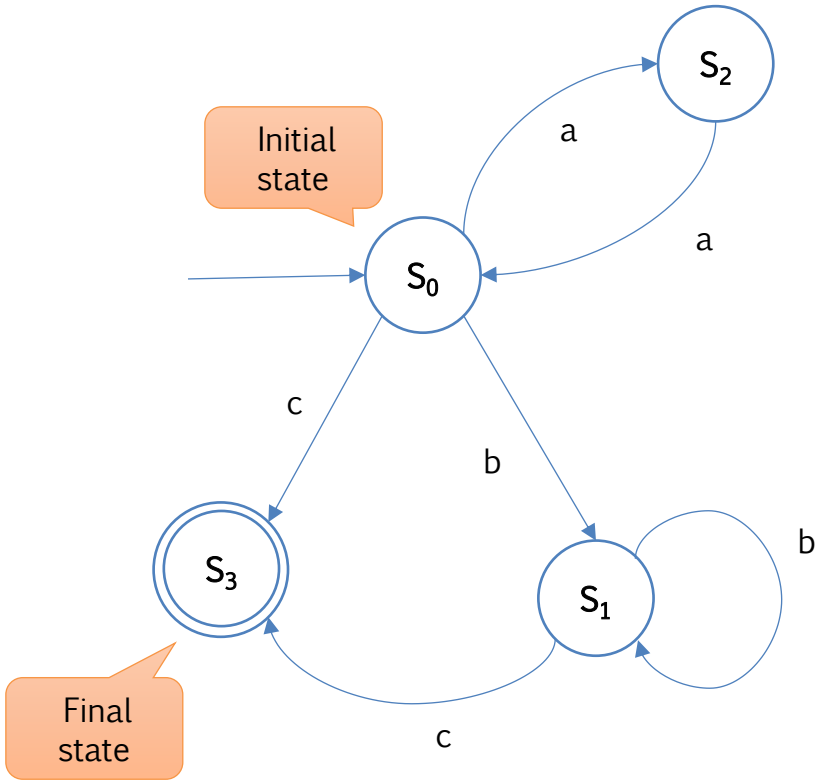
- Typically in idle state
- If you are press the button, the door opens
- You select the floor, doors close
- Then, it reaches the floor (feat. velocity control)
- Then, it opens the door, which subsequently closes after X seconds

This behavior is controlled by a **finite state automations/machine**

Finite State Automations/Machines

Problem

› Identify even sequences of *a* (even empty), followed by one, or more, or no, *b*, ended by *c*



Given an alphabet *V*,

...that identifies a language (we'll see)..

define FSA as

› S: a non-empty states set

› $s_0 \in S$: initial state

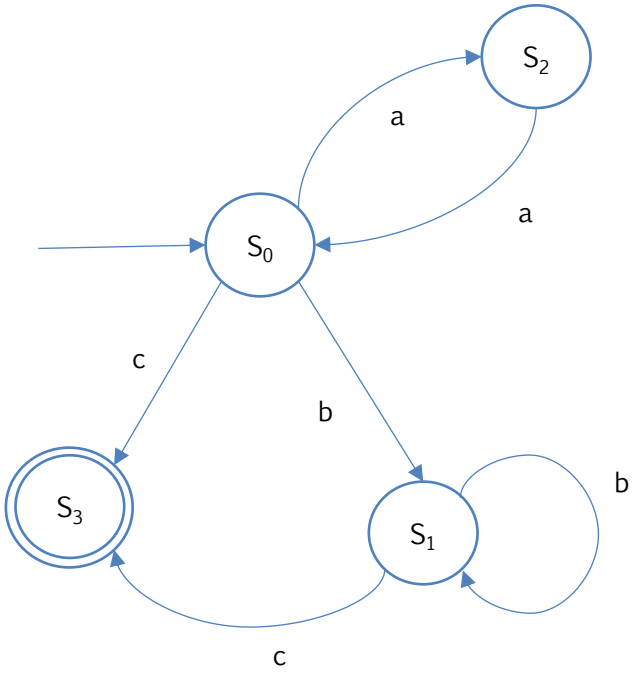
› $S_f \subseteq S$: final states set

› $t: S \times V \rightarrow S$: states transaction func

FSMs and languages

Let V* = *{ v, w, … }* contain all the combinations of words using V symbols

- Including the empty word ε
- For instance, ac, aabbc, abbabbbc belong to V*
- (note that, we can associate words in be to inputs, or combination of them)



A **language** L is a subset of V*

(abbabbbc does **not** belong to L, as previously defined)

"Identify even sequences of a (even empty), followed by one, or more, or no, b, ended by c"

State transaction function

- $sfn(s_0, b) = s_1 \quad | \quad s_0 \xrightarrow{b} s_1$
- s_y is <u>reachable</u> by s_x if there exists a path from s_x to s_y
  - a combination of alpabet symbols I (letters in our case), aka: token

$sfn: S \times V \rightarrow S$

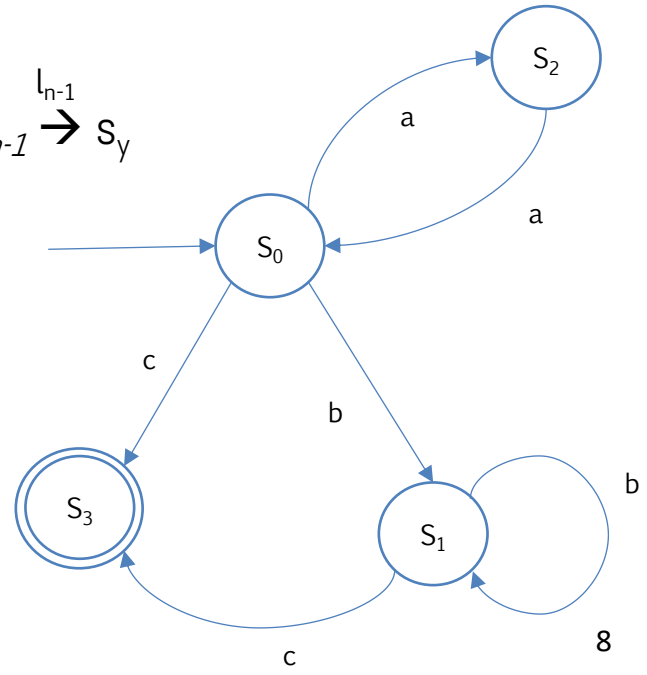
$$\rightarrow^* : \; S \times V^* \times S : \; s_x \xrightarrow{w^*} s_y$$

iff

$$w = l_1 l_2 \ldots l_n \qquad \exists \; s_t, s_u, \ldots, s_n : s_x \xrightarrow{l_1} s_t \xrightarrow{l_2} s_u \ldots s_{n-1} \xrightarrow{l_{n-1}} s_y$$

$$s_2 \xrightarrow{w^*} s_1$$

$$w = aaab \qquad \exists \; s_1, s_2, \ldots, s_n : s_2 \xrightarrow{a} s_0 \xrightarrow{a} s_2 \xrightarrow{a} s_0 \xrightarrow{b} s_1$$



Exercise

Let's code!

Implement the automata that understands whether a words is from L

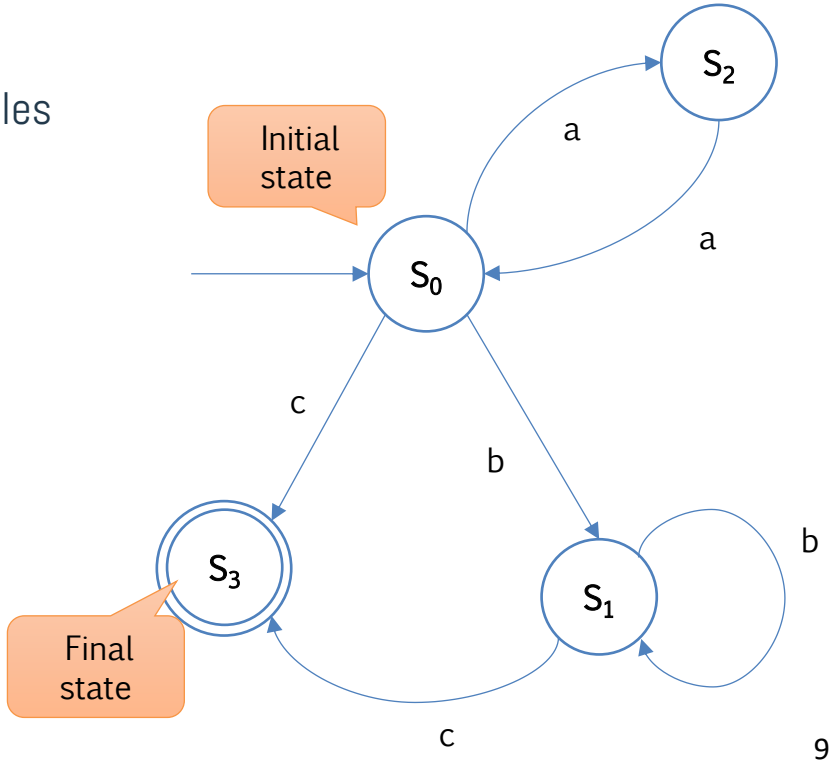
"Identify even sequences of a (even empty),

followed by one, or more, or no, b, ended by c"

Use the language that you want

- You just need IFs, CASE-SWITCH, recursion, tables
- Receive the target word from stdin
- Hint: start simple…

What's missing?



Exercise

Let's code!

Implement the automata that understands whether a words is from L

"Identify even sequences of a (even empty),

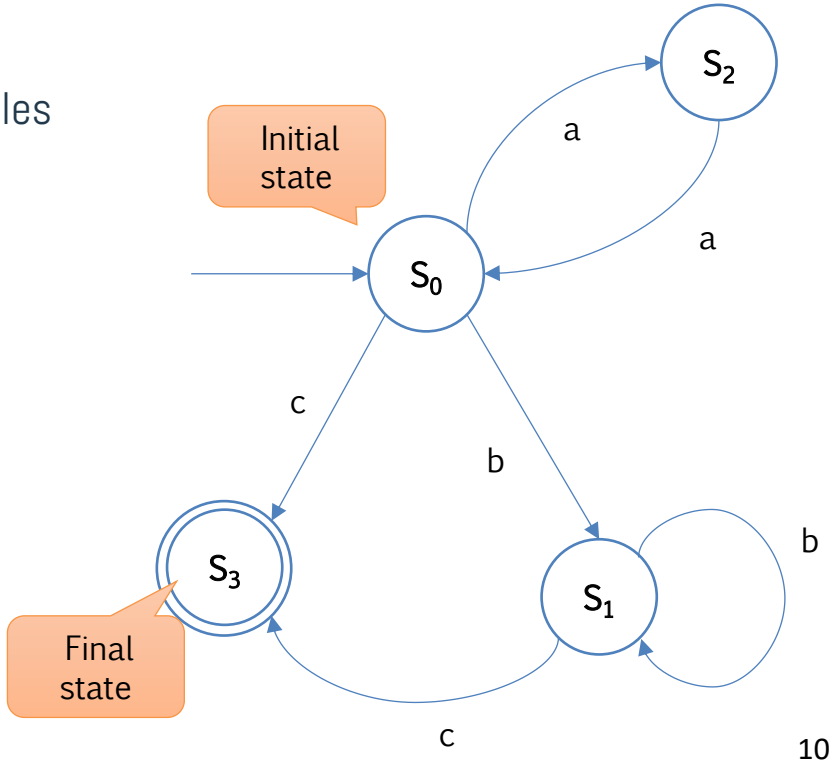
followed by one, or more, or no, b, ended by c"

Use the language that you want

- You just need IFs, CASE-SWITCH, recursion, tables
- Receive the target word from stdin
- Hint: start simple…

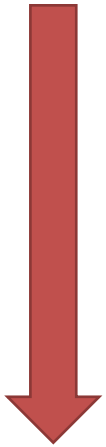
What's missing?

- In case of error => default error state
- Typically implicit in state diagrams



Hierarchy of machine types

- Base (combinatorial) machine
- Finite state machines – FSM
- FSM with stack (PDA)
- Turing machine



Base combinatorial machine

< I, O, mfn >

I : (finite) set of Input symbols

O : (finite) set of output symbols

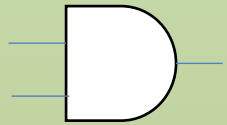
mfn: I → O machine function

- E.g., Logical ports, gates
- Suitable for continuous control
- Non suitable if you need state/memory
  - Need to model all possible cases!

Example: logical AND

I = { {0,1} × {0,1} }

O = {0,1}



mfn defined by a table

	0	1
0	0	0
1	0	1

Finite state machine

< I, O, S, mfn, sfn >

I : (finite) set of Input symbols

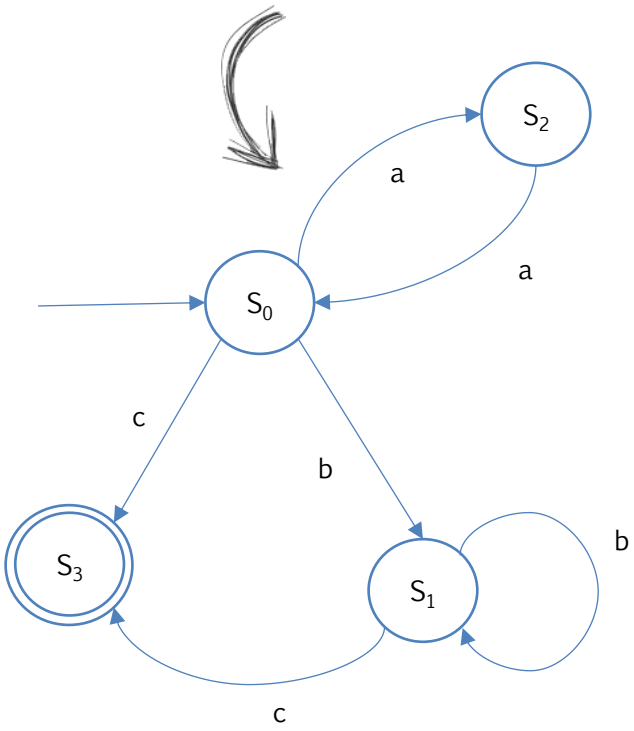
O : (finite) set of output symbols

S : (finite) set of states

mfn: I x S → O machine function

sfn: I x S → S state function

- Partly already seen
- Has memory
  - Memory is a limitation
- Here, I show only the SFN



Finite state machine with stack

< I, O, A, S, mfn, sfn >

I : (finite) set of Input symbols

A : (finite) set of stack alphabet symbols

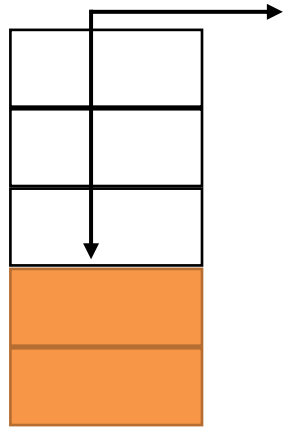
O : (finite) set of output symbols

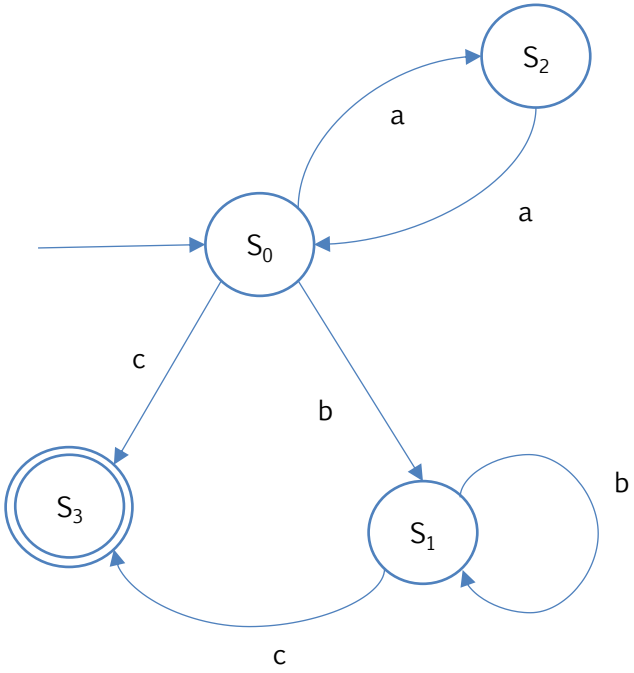
S : (finite) set of states

mfn: I x S x A → O machine function

sfn: I x S x A → S state function

- Also known as Push-Down Automata (PDA)
- Uses a stack
- We'll see them…





Turing machine

< A, S, mfn, sfn, dfn >

A : (finite) set of in/out symbols

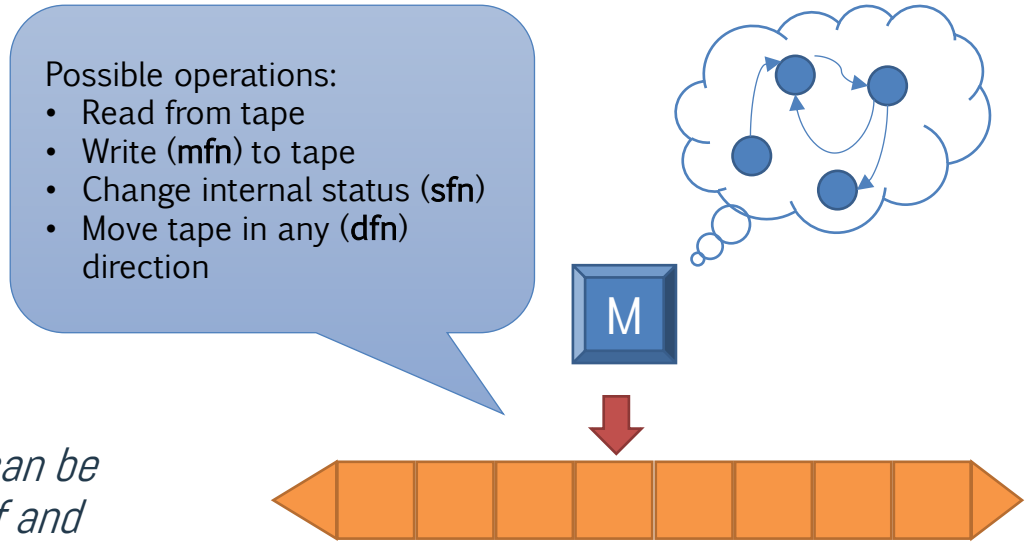
S : (finite) set of states

mfn: A x S → A machine function

sfn: A x S → S state function (inc. HALT)

dfn: A x S → { left, right, none }
direction function

› Unlimited memory



Church-Turing thesis

A function on the natural numbers can be calculated by an effective method if and only if it is computable by a Turing machine

A Universal Turing Machine

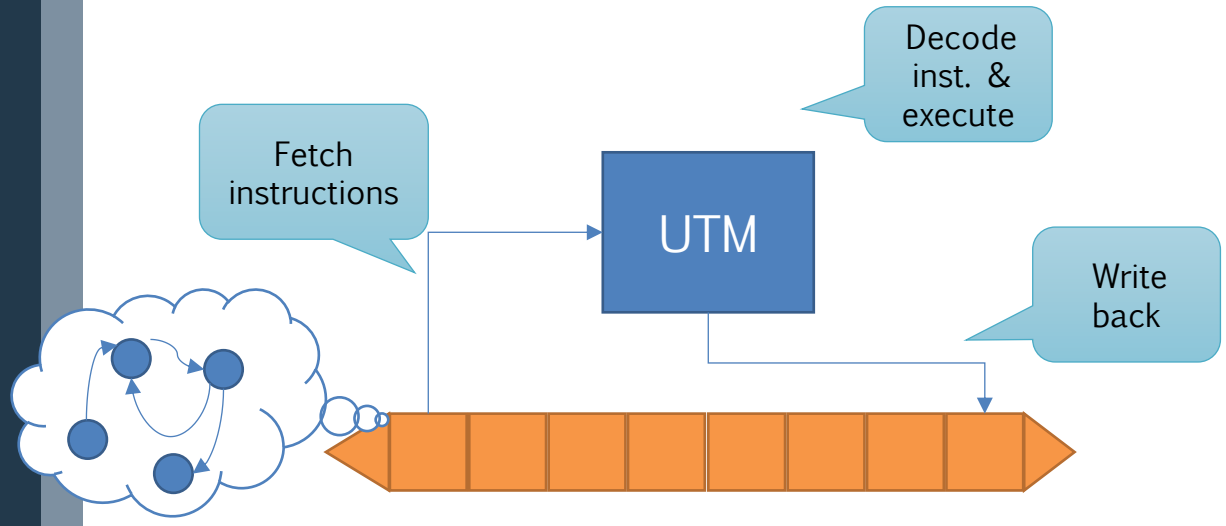
- In TM, the algorithm is inside the machine M, we write results in the tape

What if instruction as well is **in** the tape?

- We have a programmable machine, with a memory
- …..does this remind something?

Which are the catch? What do we miss?

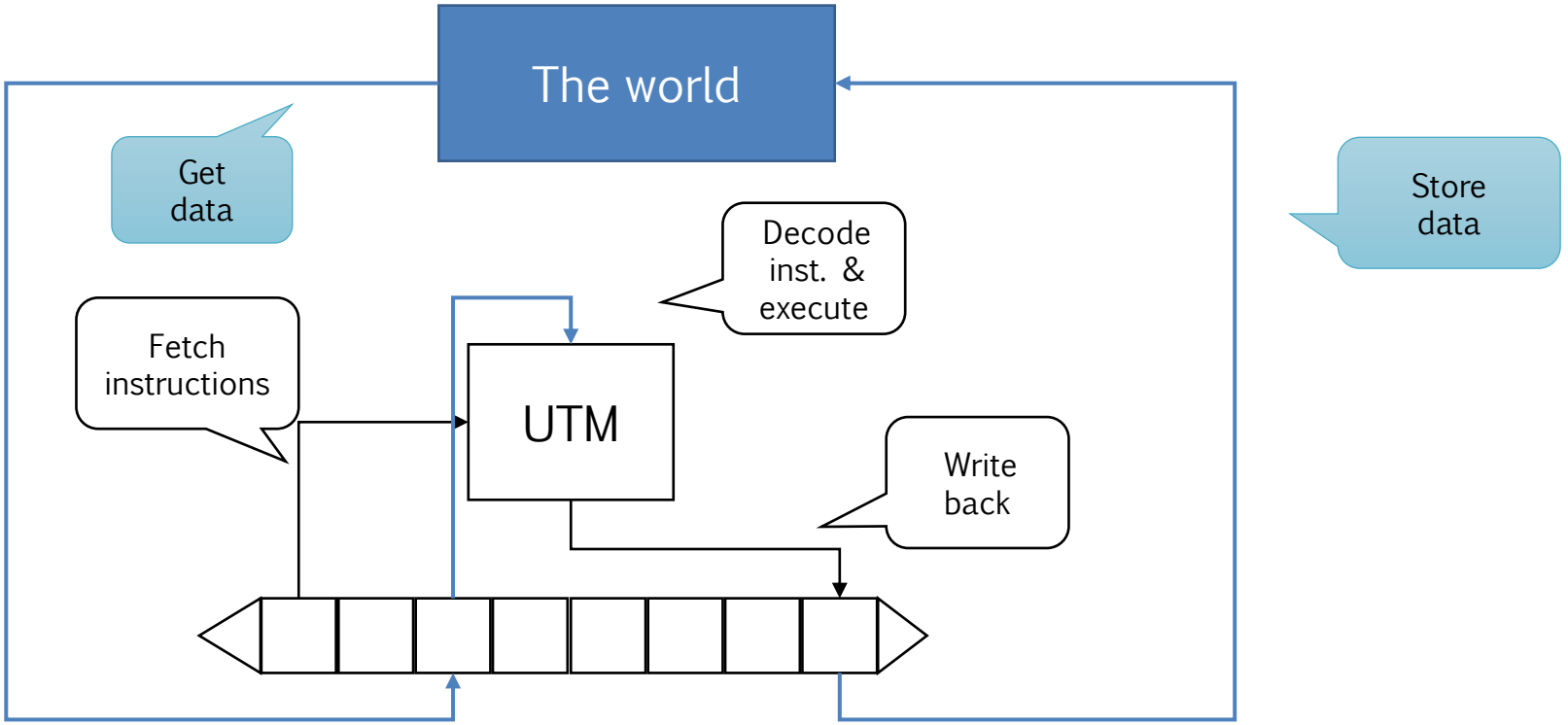
- Ok, the infinite tape makes it infeasible
- ..but what else?



The Van Neumann Machine

We also need to model the interaction with the environment!

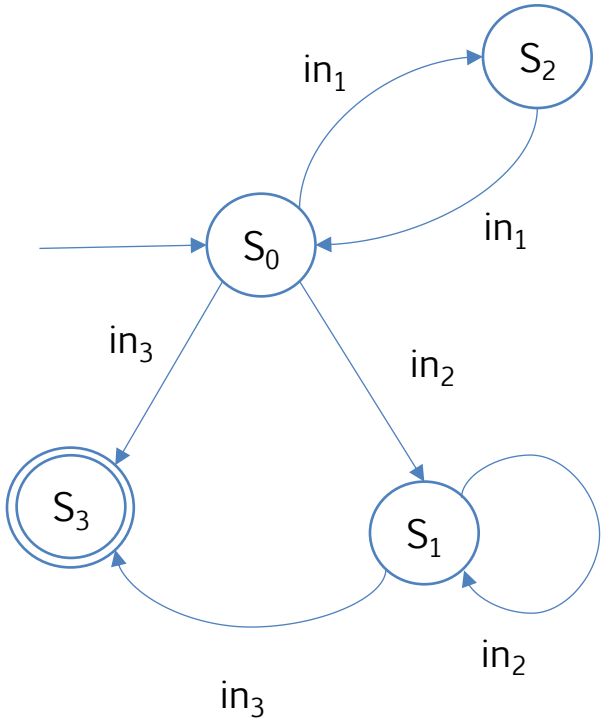
- Aka: I/O (HD/SSD is also I/O)
- Where data comes from!
- It is a real machine: we can **build** it



How to implement a FSM

A generic FSM

- Till now, we only saw machines that can recognize a **word** from a language
  - I say "word", you might want to understand "sentence"
- Let's now see how a machine can actually **produce** an output



The Machine of Mealy

› When crossing an edge, produce an output

< I, O, S, mfn, sfn >

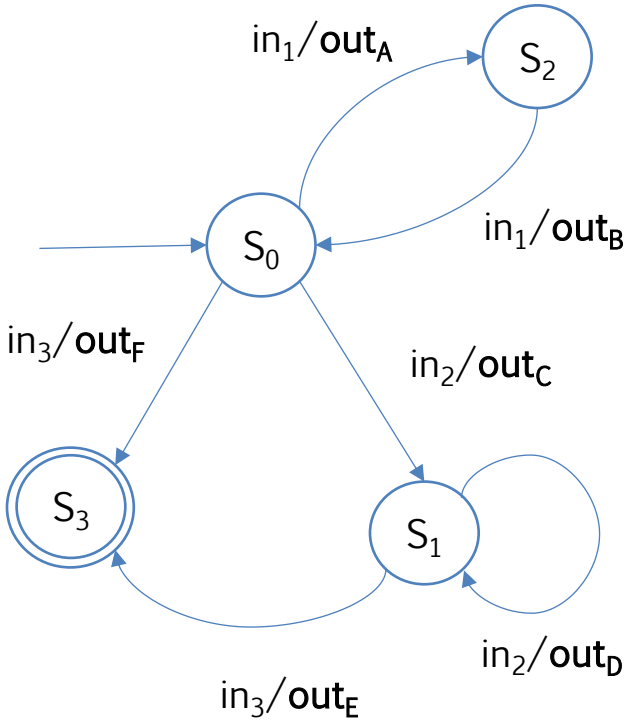
I : (finite) set of Input symbols

O : (finite) set of output symbols

S : (finite) set of states (s_0 initial state)

mfn: I x S → O machine/output function

sfn: I x S → S state transition function



The Machine of Moore

- When in a state an edge, produce an output

< I, O, S, mfn, sfn >

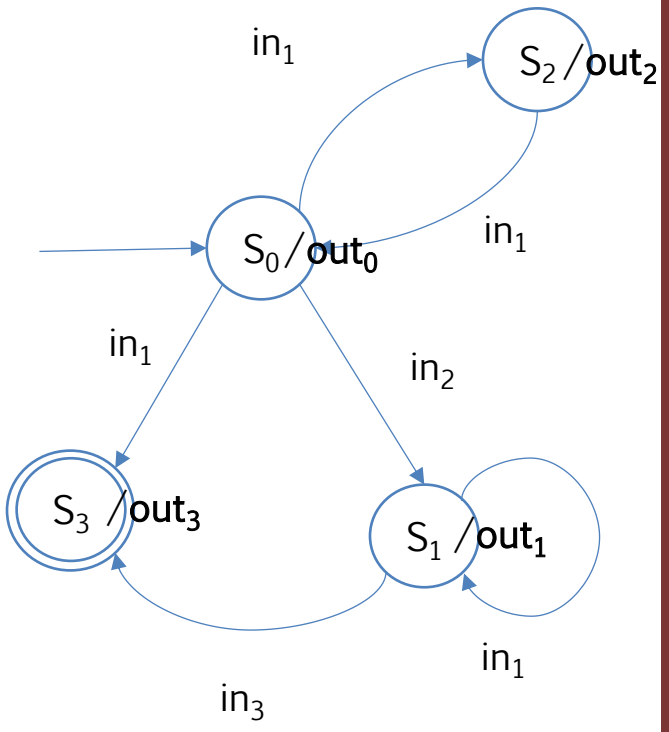
I : (finite) set of Input symbols

O : (finite) set of output symbols

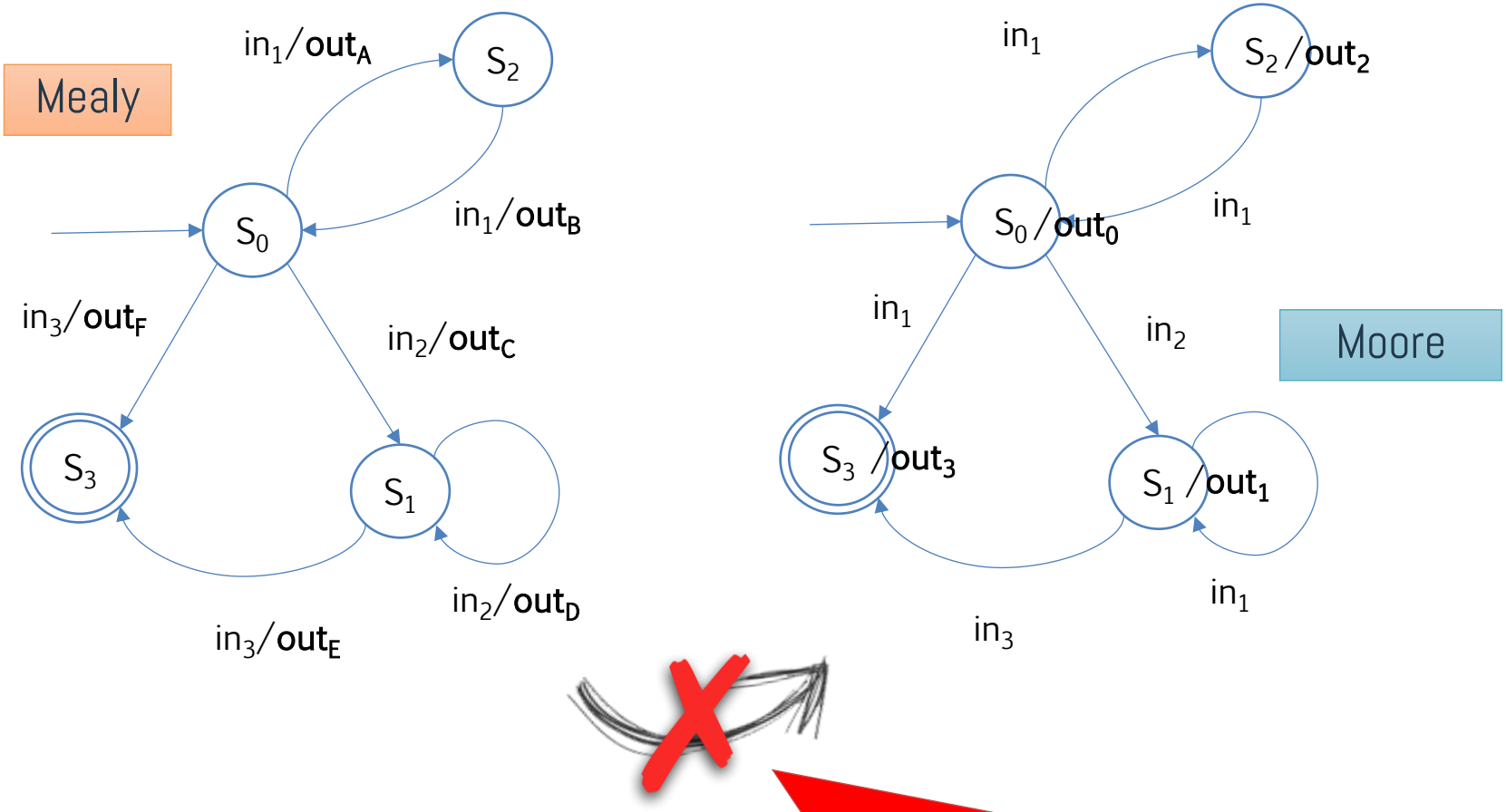
S : (finite) set of states (s_0 initial state)

mfn: S → O machine/output function

sfn: I x S → S state transition function



What's the difference?



Note: both these machines (Mealy/Moore), have an equivalent machine of the other type, but the two machines here are NOT equivalent!!

What's the difference?

Mathematically equivalent

› One can be transformed in another

..but..

› Mealy can potentially have different outs, to different inputs/transitions
  – Less states, if output depends on inputs one can add an edge to the machine
› Moore potentially keeps the output stable for all the state
  – Moore requires more states, in case out depends on input and not only on state

Exercise

Let's code!

› Implement the automata that understands whether a words is from L

"Identify even sequences of a (even empty),

followed by one, or more, or no, b, ended by c"

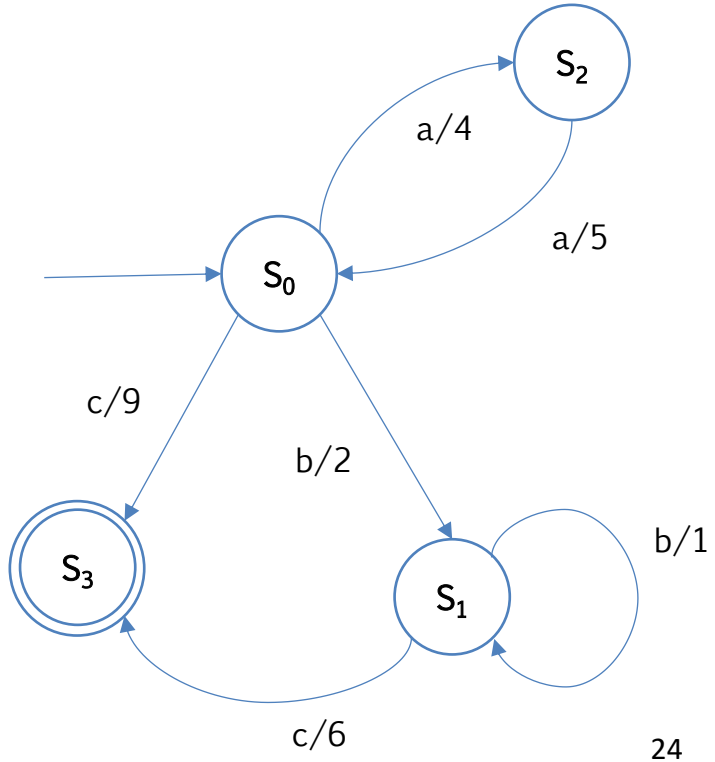
› ..and writes the corresponding number (I choose them randomly)

› Mealy? Moore? You choose

– Here, I show Mealy

Hint

› If not already done, use tables for state/output transactions



Event driven Systems

Event driven systems

A system that reacts from external stimula

- Instantly?
- Aka: Cyber-Physical Systems (CPS)

Can be

- Synchronous
- Asynchronous

Synchronous (Active polling)

- Infinite loop

```
char c;
while (c != EXIT_VALUE)
  c = readC();

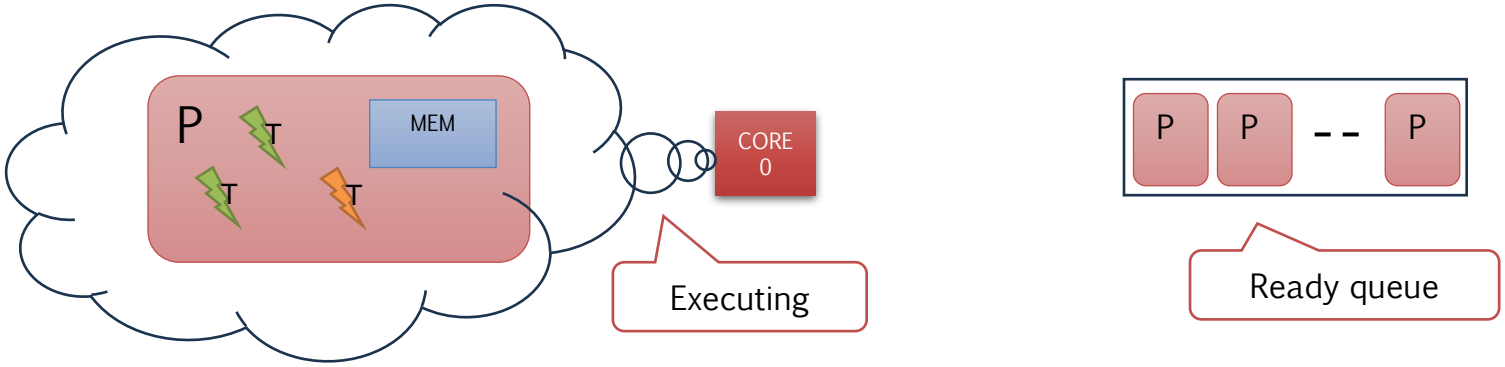
// We can go, now
```

- Pros: extremely fast and reactive
- Cons: waste of resources as one core is busy
  - Possible workaround: insert a sleep

OS scheduling in a nutshell

Modern Operating Systems (OSses) allocate processes to cores

- Aka: threads, tasks…there is a bit of confusion (we'll clarify this later on)
- Scheduling theory – Real-Time systems (not covered in this course)
- So, when a process is running, that core is busy and cannot be used by other cores



…but when is the core released?

- Can be enforced (whatchdogs, timers…) – PRO: prevents starvation | CON: drastic event
- Can be done by the process itself – PRO: done exactly when programmers want | CON: explicit in code

Yield/sleep: explicit releasing control

Typically, two primitives exposed at the application level. Example in C on GNU/Linux

› `yield ()`

› `sleep (long ms)`

```
char c;
while (c != EXIT_VALUE) {
  sleep(3000L); // Deschedule thread/process,
                // and put it in the READY Q
                // after 3000 ms
  c = readC();
}
```

```
char c;
while (c != EXIT_VALUE) {
  yield(); // Deschedule thread/process,
           // and put it in the READY Q
  c = readC();
}
```

syscalls: “explicitly” releasing control

System calls – Requesting services / help from the OS

- They are **not** library calls!
- They might require time / blocking
- They typically include yield/async waiting

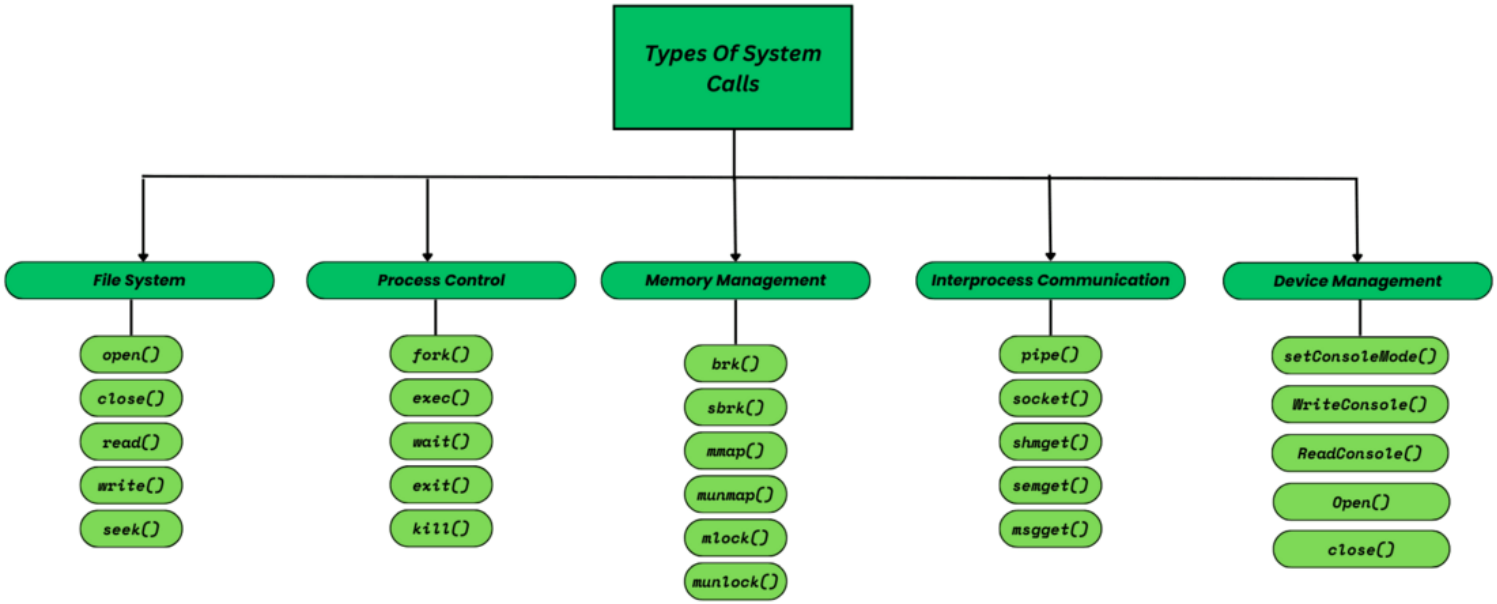
```c
FILE *f; // Declare handler (pointer) for file

f = fopen ("fileopen","mode"); // Open file

read (f,"%d", &num); // Read an integer

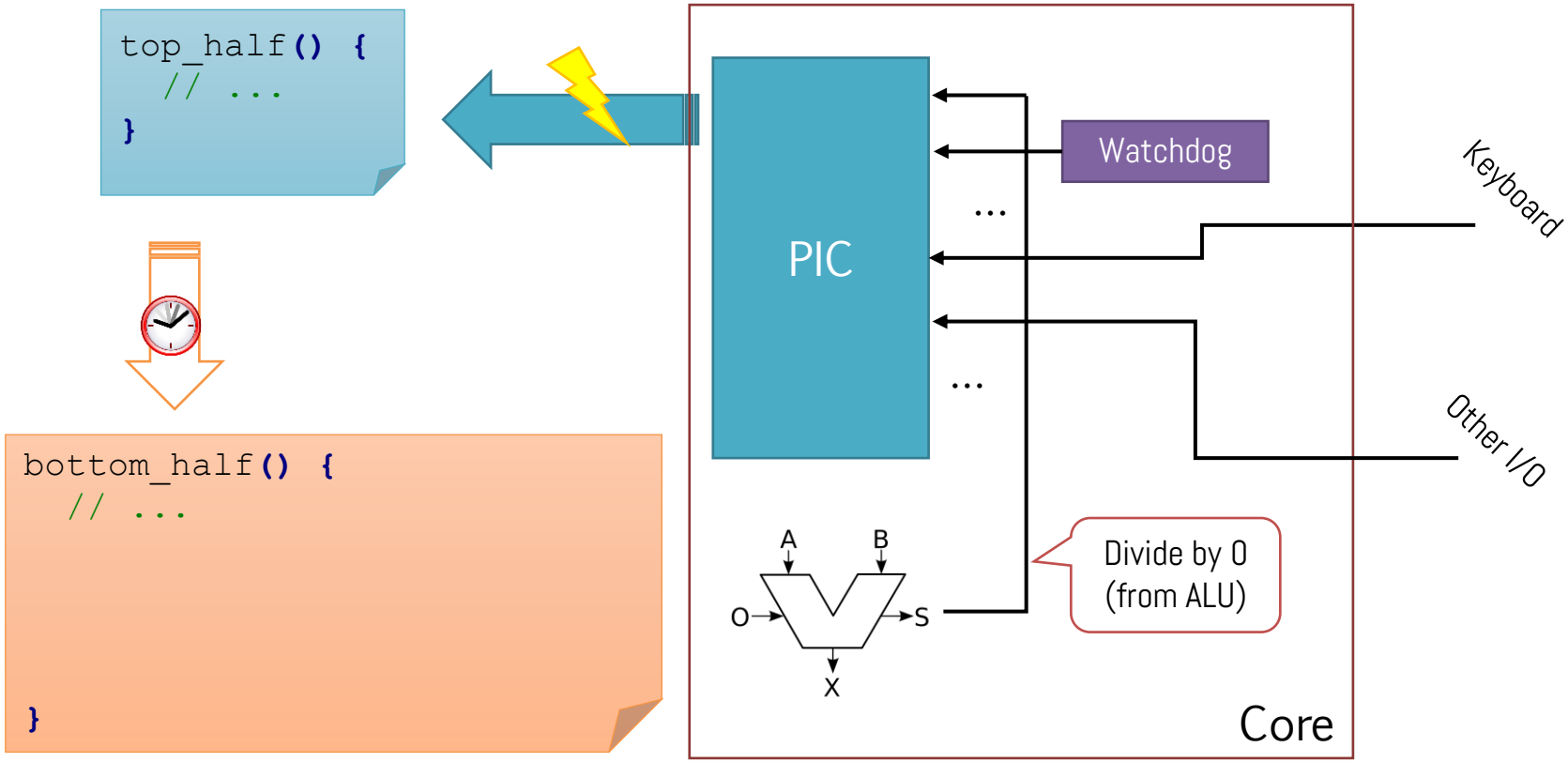
write (f, 'p'); // Write 'p' on file

fclose (f); // Close file
```



Asynchronous (Interrupt Service Routine)

› Programmable interrupt controller (hierarchy)



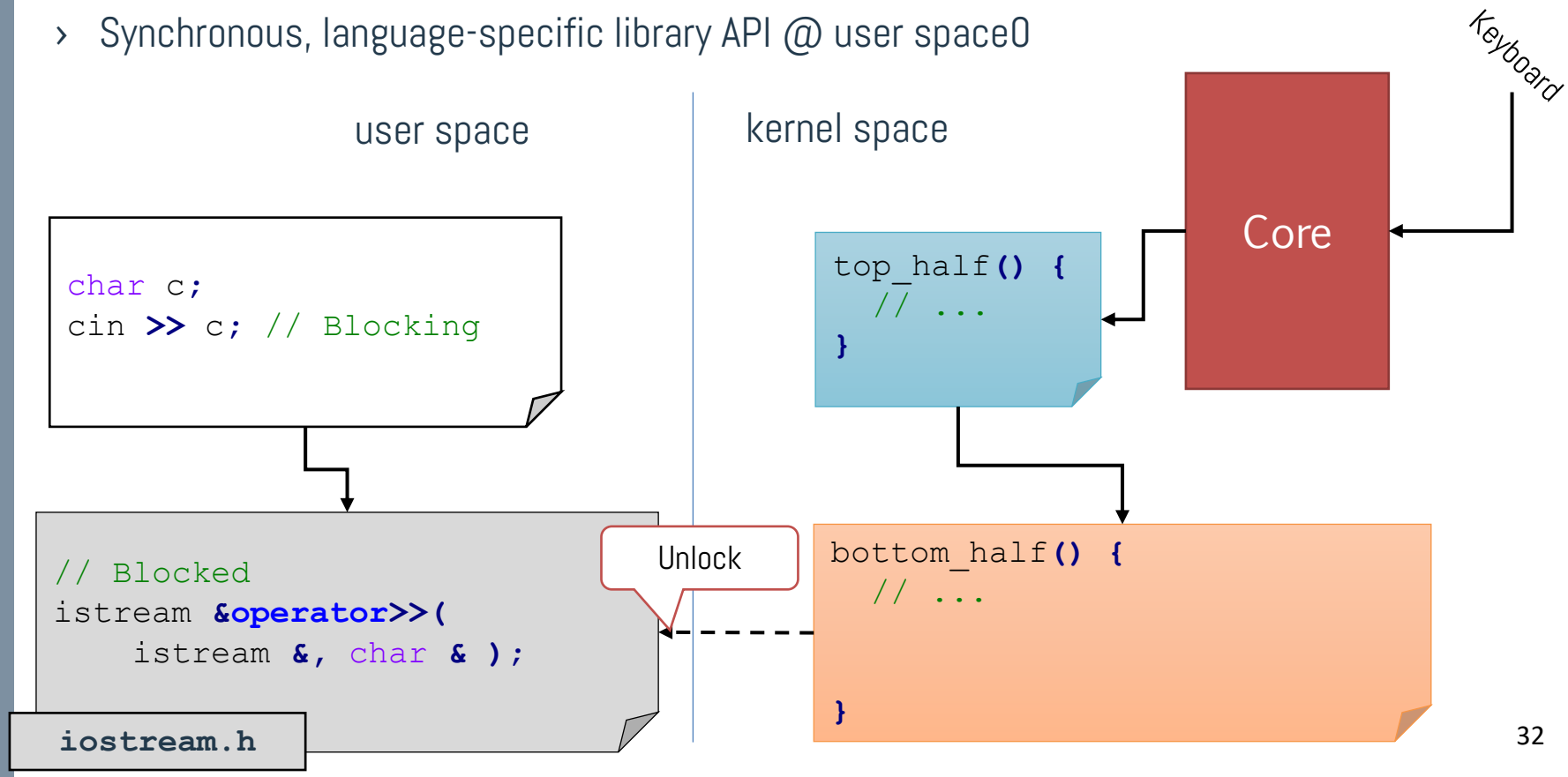
› Pros: "pay-as-you-go"

› Cons: takes more time to issue a ISP

...a mix of the two

Keyboard management in a General-Purpose system

- GNU/Linux
- ISP with bottom-half and top-half @ kernel space
- Synchronous, language-specific library API @ user space0

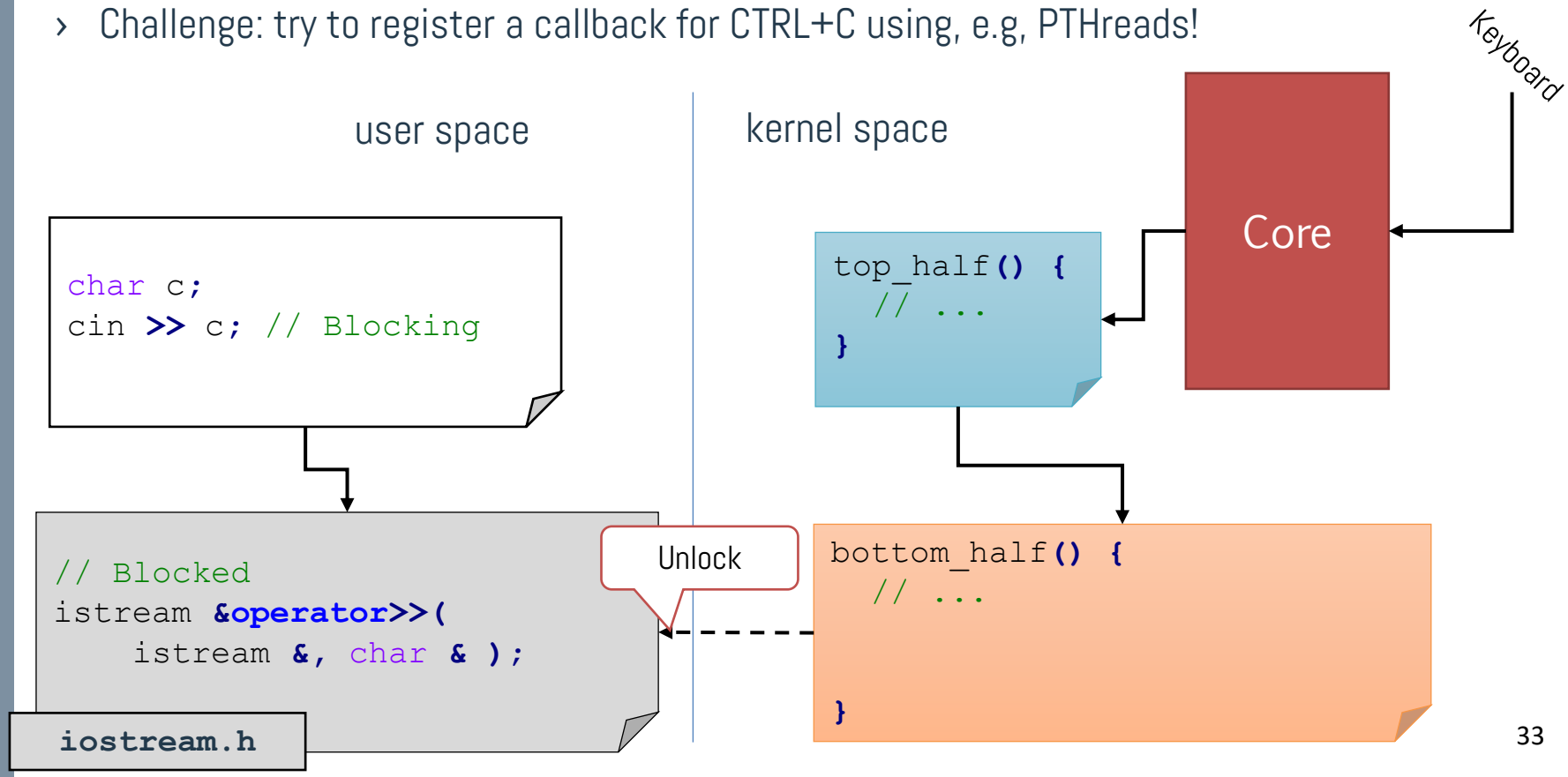


Asynch execution - small recap

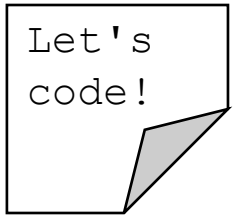
- **Pros**: avoid starvation and waste of computational resources
- **Cons**: less reactive; potentially requires explicit user control
  - E.g., to register callback code, i.e., *"what has to be done when the event X triggers?"*

In this example, keyboard I/O is entirely handled by the OS, no callback is required

- Challenge: try to register a callback for CTRL+C using, e.g, PTHreads!



How to run the examples



› Find them in `Code/` folder from the course website

For C++: compile

```
› $ gcc code.cpp -o code -Wall -lstdc++
```

Run (Unix/Linux)

```
$ ./code
```

Run (Win/Cygwin)

```
$ ./code.exe
```

References



Course website

- http://hipert.unimore.it/people/paolob/pub/Industrial_Informatics/index.html

My contacts

- paolo.burgio@unimore.it
- http://hipert.mat.unimore.it/people/paolob/

Resources

- Alessandro Fantechi, «Informatica Industriale», Città Studi Edizioni
- For interrupts
  - Robert Love, «Linux kernel development», Pearson
- For GNU Bison
  - http://dinosaur.compilertools.net/bison/bison_5.html
- A "small blog"
  - http://www.google.com